

Sistema de Cadastro de Clientes e Funcionários

Documentação técnica do projeto — Python + SQL (SQLite)

1. Sobre o projeto

Este documento descreve o sistema desenvolvido, as decisões tomadas na construção dele, a estrutura do banco de dados e os testes realizados para validar o funcionamento. O objetivo foi criar um sistema simples, no nível de um projeto de faculdade (3º período de Ciência da Computação), usando apenas recursos padrão do Python (sem frameworks externos), para ficar fácil de entender, rodar e continuar evoluindo por conta própria.

2. Tecnologias utilizadas

- **Python 3** — linguagem principal do sistema.
- **SQLite (módulo `sqlite3`)** — banco de dados relacional, já incluso no Python, sem necessidade de instalar servidor de banco de dados separado.
- **unittest** — biblioteca padrão de testes automatizados do Python.
- **re (expressões regulares)** — usada nas validações de CPF e e-mail.

3. Estrutura do banco de dados

O banco de dados (arquivo **empresa.db**, criado automaticamente na primeira execução) possui duas tabelas:

Tabela: clientes

Coluna	Tipo	Descrição
id	INTEGER (PK, autoincrement)	Identificador único do cliente
nome	TEXT	Nome completo (obrigatório)
cpf	TEXT (único)	CPF do cliente, sem duplicidade
email	TEXT	E-mail (opcional)
telefone	TEXT	Telefone de contato

Tabela: funcionarios

Coluna	Tipo	Descrição
id	INTEGER (PK, autoincrement)	Identificador único do funcionário
nome	TEXT	Nome completo (obrigatório)
cpf	TEXT (único)	CPF do funcionário, sem duplicidade
cargo	TEXT	Cargo/função

salario	REAL	Salário (não pode ser negativo)
data_admissao	TEXT	Data de admissão (dd/mm/aaaa)

4. Funcionalidades implementadas

Para **Clientes** e para **Funcionários**, o sistema oferece um CRUD completo via menu no terminal:

- Cadastrar (com validação de nome, CPF e e-mail/salário)
- Listar todos os registros, em ordem alfabética
- Buscar por nome (busca parcial, tipo 'contém')
- Atualizar dados de um registro pelo ID
- Remover (deletar) um registro pelo ID

5. Validações aplicadas

- **Nome:** não pode ser vazio.
- **CPF:** precisa ter 11 dígitos numéricos (pontos e traços são ignorados); além disso, o banco de dados impede CPF duplicado (constraint UNIQUE).
- **E-mail:** se preenchido, precisa ter o formato básico algo@algo.algo (é opcional).
- **Salário:** precisa ser um número válido e não pode ser negativo.

6. Testes realizados

Foram criados **13 testes automatizados**, usando o módulo **unittest** (arquivo test_sistema.py). Cada teste roda em um banco de dados separado (teste.db), criado do zero antes de cada teste e apagado logo depois, para garantir que um teste não interfira no outro.

6.1 Lista de testes e o que cada um verifica

Teste	O que verifica
test_cpf_valido	CPF com 11 números é aceito; CPF incompleto ou com letras é rejeitado
test_email_valido	E-mail correto é aceito; vazio é permitido; sem '@' é rejeitado
test_cadastrar_cliente_sucesso	Cliente válido é salvo corretamente no banco
test_cadastrar_cliente_cpf_invalido	Cadastro é bloqueado quando o CPF é inválido
test_cadastrar_cliente_cpf_duplicado	Sistema impede dois clientes com o mesmo CPF
test_buscar_cliente_por_nome	Busca parcial por nome encontra e não encontra corretamente
test_atualizar_cliente	Dados do cliente são atualizados no banco
test_deletar_cliente	Cliente é removido do banco corretamente
test_cadastrar_funcionario_sucesso	Funcionário válido é salvo corretamente
test_cadastrar_funcionario_salario_invalido	Bloqueia salário que não é número
test_cadastrar_funcionario_salario_negativo	Bloqueia salário negativo

test_atualizar_funcionario	Cargo e salário são atualizados no banco
test_deletar_funcionario	Funcionário é removido do banco corretamente

6.2 Resultado da execução dos testes

Comando executado: `python -m unittest test_sistema -v`

```
test_atualizar_cliente ... ok
test_atualizar_funcionario ... ok
test_buscar_cliente_por_nome ... ok
test_cadastrar_cliente_cpf_duplicado ... ok
test_cadastrar_cliente_cpf_invalido ... ok
test_cadastrar_cliente_sucesso ... ok
test_cadastrar_funcionario_salario_invalido ... ok
test_cadastrar_funcionario_salario_negativo ... ok
test_cadastrar_funcionario_sucesso ... ok
test_cpf_valido ... ok
test_deletar_cliente ... ok
test_deletar_funcionario ... ok
test_email_valido ... ok

-----
Ran 13 tests in 0.040s

OK
```

6.3 Teste manual (demonstração no terminal)

Além dos testes automatizados, o sistema também foi testado manualmente, chamando as funções principais diretamente, simulando o uso real:

```
>>> cadastrar_cliente('Felipe Souza', '11144477735', 'felipe@email.com',
'27999998888')
(True, 'Cliente cadastrado com sucesso!')

>>> cadastrar_cliente('Teste', '123', 'teste@email.com', '')
(False, 'CPF inválido. Deve conter 11 dígitos.')

>>> cadastrar_funcionario('Ana Paula', '22255588846', 'Analista de TI', '4200')
(True, 'Funcionário cadastrado com sucesso!')

>>> listar_clientes()
[(1, 'Felipe Souza', '11144477735', 'felipe@email.com', '27999998888')]

>>> listar_funcionarios()
[(1, 'Ana Paula', '22255588846', 'Analista de TI', 4200.0, '16/08/2026')]
```

7. Como executar o projeto

- Ter o Python 3 instalado (não precisa instalar nenhuma biblioteca extra para rodar o sistema, só para gerar este PDF).
- Rodar o sistema: `python main.py`
- Rodar os testes: `python -m unittest test_sistema -v`

8. Limitações e possíveis melhorias (fica por sua conta)

O sistema foi feito de forma simples, focando em CRUD + validação + testes. Como próximos passos, dependendo do que for pedido no projeto, dá pra evoluir com:

- Trocar o SQLite por PostgreSQL/MySQL, caso o projeto exija um SGBD 'de verdade' em rede.
- Criar uma interface gráfica (Tkinter) ou uma versão web (Flask/Django).
- Adicionar autenticação de usuário (login) e níveis de permissão.
- Gerar relatórios em PDF/Excel diretamente pelo sistema.
- Adicionar validação de CPF mais rigorosa (dígitos verificadores).
- Criar relacionamento entre tabelas (ex: vincular vendas a clientes).

9. Arquivos entregues

- **main.py** — sistema principal (banco de dados, CRUD, menus).
- **test_sistema.py** — 13 testes automatizados (unittest).
- **Documentacao_Sistema_Cadastro.pdf** — este documento.